

Archiving Years of Dataverse Audit History Before You Prune It — A Pragmatic, Open-Source Pattern

A note up-front. This post documents an architectural choice — not a Microsoft product, not a Microsoft offering, and not officially supported. For most Dataverse-to-Azure replication needs the first-class options are [Azure Synapse Link for Dataverse](#) and [Microsoft Fabric Link for Dataverse](#); if either fits your scenario, prefer them.

For the audit table specifically, Synapse Link is now the first-choice answer. In the last year or two Microsoft added explicit support for exporting the `audit` table via the [Delta Lake profile of Azure Synapse Link for Dataverse](#) — there's even a dedicated docs page for the Power BI reporting flow on top of it. If you can run that link in your tenant, run that. This pattern is for the cases where you can't, or where what lands in the lake isn't what you actually need.

What "can't, or isn't enough" looks like in practice: (1) your analytics platform of record sits outside Azure (most commonly Snowflake) and you don't want to add Synapse + ADLS + Spark just to feed the audit table; (2) internal review hasn't approved the link in your tenant, or region/governance constraints rule it out; (3) you need the *decoded* `RetrieveAuditDetails` payload — old value, new value, attribute mask, related-record diffs — rather than the packed `changedata` column that lands in the lake unparsed; (4) you need the cold archive to *outlive* the source rows, i.e., you want to safely prune in Dataverse without losing the evidence the link would propagate the delete for. Treat this as a reference implementation you can adapt, fork, or discard.

Why the audit table is special

The **audit table** is different from the rest of Dataverse in two ways that matter for an archive design:

1. It's an immutable, append-only record of *who changed what, when, and from where* — the closest thing Dataverse has to a forensic ledger.
2. The valuable part of an audit row is not the row itself; it's the diff (old value → new value, attribute mask, related-record context). The audit row stores that diff in a packed `changedata` column, and the bound `RetrieveAuditDetails` function is what decodes it into a structured `OldValue` / `NewValue` / `ChangedAttributes` shape your downstream tools can actually query. Synapse Link with the Delta Lake profile *will* carry the `changedata` column to the lake, but you still need a parser on the other side; this pattern calls `RetrieveAuditDetails` at archive-time so what lands in the destination is already decoded and immediately queryable.

That combination makes the audit table the single most useful Dataverse table for:

- Regulatory and compliance investigations
- "Why did this opportunity status change in Q3 of 2022?" forensic queries (years after the fact)
- Internal analytics on user behaviour and process adoption

It's also the table that grows the fastest. The Dataverse default retention is 90 days, but in practice many enterprises extend that to several years — or set it to *never delete* — to retain evidence for compliance and forensic review. The result, often after five to seven years, is an audit table holding tens of GB to multiple TB of capacity, dominating the entitlement bill, and rarely accessed in normal operations.

At that point the storage conversation becomes unavoidable. The realistic choices are:

1. **Keep buying entitlement.** Predictable, but unbounded.
2. **Move the cold tail somewhere cheaper that you control, then let Dataverse's audit-deletion job reclaim the space.** The hot months stay in Dataverse where users expect them; the years of historical evidence live in your own storage account, queryable when you need them.

This pattern is for option 2 — specifically, for the *one-time bulk export of multi-year history*, with the option to keep a slow trickle running afterwards if you want to top up.

A crucial point that often gets lost: this pipeline does not need to run live. It is perfectly reasonable to be deliberately months or years behind real-time. The goal is to get a defensible copy of *cold* data out — the rows you are about to allow Dataverse to delete — not to mirror the audit feed in real time.

What "good" looks like for an external audit copy

Before showing any code, here's the rubric I held this design to. If you build your own, hold yours to the same rubric:

Property	Why it matters
Idempotent	Re-running the same time window must not duplicate rows. Network blips happen.
Crash-safe (exactly-once-effective)	If the process dies mid-window, the next run must replay the same window cleanly. The watermark advances <i>only</i> after the data is durable in the destination.
Bounded memory	A backlog of millions of audits cannot be loaded all at once.
Backpressure-aware	Dataverse rate-limits aggressively. Throttle responses must not drop rows.
Observable	Every window logs <code>[entity] mode=BACKLOG/LIVE, lag=Nmin, window=10min, records=N</code> so you can watch it work.
Sink-agnostic	The "where does it land" decision is config, not code. Storage choices change; the orchestrator shouldn't.
Field-level discretion	Audit details can carry PII. The pattern should let admins narrow which attributes leave the platform.

The pattern

The pipeline is conceptually four stages, repeated per entity, per time window:

```
Per entity (account, contact, ...):  
  1. Read watermark (lastSyncEnd) from sink's state container  
  2. Compute next time window: [lastSyncEnd, lastSyncEnd + N min)  
  3. Fetch audit headers from Dataverse Web API:  
     $filter=createdon ge <start> and createdon lt <end>  
  4. For each header → call RetrieveAuditDetails (bound function)  
  5. (Optional) Project to allow-listed attributes only  
  6. Upsert each audit doc into sink (id = auditid, idempotent)  
  7. Advance watermark ONLY if step 6 fully succeeded  
  
Loop until caught up; then either exit (function) or sleep (daemon)
```

Three details in this picture do most of the resiliency work. They are deceptively simple:

Detail 1: Half-open time windows (ge / lt)

```
[09:00:00, 09:10:00) → window 1  
[09:10:00, 09:20:00) → window 2
```

The boundary moment (09:10:00.000) belongs to window 2, not window 1. So adjacent windows never overlap and never gap, no matter how many times you replay. This is the same trick Kafka uses for offsets — it's why you can run the loop with confidence.

Detail 2: The destination document key is the Dataverse auditid GUID

Dataverse already assigns a globally unique GUID to every audit row. That GUID becomes the document `id` in the sink. So when you upsert the same audit twice, the second write is a no-op overwrite of the first — idempotency for free, no client-side dedupe table to maintain.

Detail 3: The watermark moves *after* the write, not with it

The naive version of this pipeline does:

```
records = fetch_window(...)  
sink.write(records)           # 950/1000 succeed; 50 fail with 429  
sink.update_watermark(window_end) # ← BUG: window now "done", 50 rows lost forever
```

The resilient version raises a typed exception when *any* record fails, and the watermark update is conditional on a clean write:

```
try:
    written = sink.write(records)
    sink.update_watermark(window_end)
except SinkPartialWriteError:
    # Watermark stays put. Next loop iteration replays the same window.
    # Rows that already landed become no-op upserts. Rows that failed get retried.
    raise
```

This single change is the difference between "best effort" and "exactly-once-effective." It's also the mistake most often made when people roll their own.

Going deeper. The exact API call shape, why concurrency is bounded globally instead of per-entity, the state-container schema, the half-open math, the worked test runs (smoke / partial-failure drill / full-backlog harness), and the step-by-step "how to add a sink for your own destination" guide all live in the repo's design notes:

[unified-deployment/DESIGN.md](#)

The intent is to keep the blog itself focused on the *shape of the problem and the choice of pattern*; the depth that matters when you actually fork the code lives next to the code.

Choosing where it lands

The orchestrator is sink-agnostic — it talks to a single `AuditSink` interface (`get_state` , `update_state` , `write_audits`) and the destination is a config switch, not a code change. The reference implementation ships with four production-shaped sinks plus a no-op for testing. None of them is *the* answer; they map to platforms enterprises already operate:

Sink	When to consider it
Azure Cosmos DB (NoSQL API)	Operational lookups — "show me everything user X did to record Y in 2022" in milliseconds. Hierarchical partition keys (<code>/entity</code> + <code>/auditYearMonth</code>) keep partitions small as the archive grows over years. Document TTL doubles as a retention policy if you want one. Serverless mode suits a slow-trickle archive workload.
Azure Data Lake Storage Gen2 (Parquet)	The cheap-cold-storage option. Years of audit history land as partitioned Parquet files (<code>entity=.../year=.../month=.../</code>), readable from Fabric notebooks, Synapse Serverless SQL, Databricks, or any Parquet engine. Costs scale with bytes, not throughput — ideal when the archive is rarely queried but must exist.
OneLake (Parquet)	Same Parquet shape as ADLS, but landed inside a Microsoft Fabric Lakehouse. Immediately queryable from a Fabric SQL endpoint, notebooks, and Power BI without further plumbing. The natural choice if your downstream BI is Fabric.
Snowflake (MERGE INTO)	The natural choice when Snowflake is already the analytics platform of record and adding a separate Microsoft analytics estate just for audit data isn't on the table. <code>MERGE INTO ... ON audit_id</code> keeps the same idempotency contract as the Cosmos upsert, and the warehouse stays paused between archival batches.
No-op (logs only)	First-day connectivity testing. Confirms the Dataverse side works before you provision any storage.

A reasonable default split many enterprises arrive at:

- **ADLS Gen2 / OneLake** (or **Snowflake**, if that's your platform) holds the durable historical archive — cheap, partitioned, queryable when (rarely) needed.
- **Cosmos DB** holds the most recent N months for fast operational lookup if there is a use case for it; otherwise skip it entirely.

Adding a sink for storage you already own (e.g., BigQuery, Redshift, on-prem object storage) is roughly 100 lines of Python and one factory entry.

What real-world numbers will look like

The worked test run above shows what a clean sandbox run looks like. Real numbers in your tenant will vary by orders of magnitude depending on:

- How many entities have auditing enabled
- The shape of `RetrieveAuditDetails` calls (more changed attributes per row = more bytes per call)
- Dataverse Web API rate limits applicable to your environment
- Concurrency you allow (`max_concurrent_entities` in the config)
- Sink throughput (Cosmos serverless RU autoscale, ADLS upload bandwidth, Snowflake warehouse size)

The useful operational signal is not the absolute throughput — it's that the throughput is *stable* and the per-window log lines tick predictably. If they don't, look at lag, sink errors, or 429 responses from Dataverse before scaling up concurrency.

When to reach for it (and when not to)

Reasonable fit:

- You have multiple years of accumulated audit history in Dataverse and need to move the cold tail off the platform before pruning to reclaim entitlement.
- You want the *decoded* `RetrieveAuditDetails` payload (old value → new value, attribute mask, related-record context) landing in the destination ready to query — rather than the packed `changedata` column Synapse Link delivers, which still needs a parser on the consumer side.
- Your analytics platform of record sits outside Azure (most commonly Snowflake) and you don't want to add Synapse + ADLS + Spark to your stack just to land the audit table.
- Synapse Link isn't an option in your tenant — region pairing, governance review, or the cost floor of running ADLS + a Spark pool 24/7 don't fit your environment.
- You're comfortable running this as a *batch job* — once for the historical backfill, then perhaps quarterly or annually to top up — rather than as a live continuous feed. Being deliberately months or years behind real-time is fine and often desirable.
- You want field-level control over which attributes leave the platform — useful when audit details contain regulated data.

Not a good fit:

- You can run [Azure Synapse Link with the Delta Lake profile](#) and you're happy parsing the packed `changedata` column on the consumer side, *and* your destination is ADLS / Synapse / Power BI. That's the supported, first-class path for the audit table — use it.
 - You only need *current state* of business tables (account, contact, opportunity). Use Synapse Link / Fabric Link — they do exactly that and you don't need this pattern.
 - You need sub-second freshness in the destination. The pattern's natural cadence is one window length (10 min in the reference config); for true real-time, use Dataverse webhooks or change-tracking APIs.
 - You don't have somewhere to operate a small Python container, function, or scheduled job — even an annual one.
 - You don't have an internal owner who can be paged when the schedule fails.
-

The reference implementation

The code that backs this post lives at <https://github.com/SweetsNSavories/DataverseAuditLogSyn> under MIT, with no warranty. The `unified-deployment` folder is the version this post describes — single Python codebase, swap sinks via `config.json`, runs locally / in a container / as an Azure Function.

If you want the implementation depth this post deliberately leaves out — exact API shapes, watermark math, partial-failure drill, sink-author checklist, hosting variants, observability hooks, and the full list of operational responsibilities a self-hosted export carries — it all lives in one place:

- [unified-deployment/DESIGN.md](#)

Issues, forks, and pull requests welcome via the repo.